

VU MINH HIEU

Hanoi, Vietnam | vuminhhieu2412@gmail.com | LinkedIn

Attachment 2: Project Sample, Contaminated Land MCP

AI Engineer (hands-on), Western Environmental Pty Ltd

Link: <https://github.com/ghiaog123/contaminated-land-mcp>

What it is

Contaminated Land MCP is a Python MCP server (FastMCP, stdio) with four tools, built end to end with Claude Code. It runs in Claude Desktop, Claude Code or any MCP client, so an environmental consultant works in plain English and the host model picks the tools. The server turns the three jobs of a contaminated-land assessment into tools: screening lab results against published criteria, finding the right page in thousands of pages of guidance, and drafting the results section of a report. Only `draft_section` calls a model; the other three are plain code.

Your problem, the tool, the measured result

Your need	Tool that handles it	Measured result
Structure laboratory data and field records	<code>screen_lab_results</code> : lab CSV compared with the criteria table in plain Python; anything not comparable is listed under <code>not_screened</code>	13/13 cases exact, expected outputs hand-computed
Regulatory reporting where the format is rigid	<code>screen_lab_results</code> + <code>draft_section</code> : every exceedance cites document, page and table; every number and citation in the draft traces to its source	Citation validity 1.0 and number fidelity 1.0 across 9 drafts (3 sites x 3 draws)
Retrieval over documents nobody can search	<code>search_guidance</code> : hybrid keyword + vector search (LanceDB, rank fusion); every passage carries document, page and section	recall@5 0.85, MRR 0.80 on 20 questions over 3 documents (1,712 chunks)
Guard rails around client and project data	Redaction inside <code>draft_section</code> : client names and site addresses become placeholders before the model sees them, restored in the returned draft	0 PII leaks
LLM systems people use, tool use, agentic workflows	All four tools, driven by the host model over MCP	Right tools and arguments on 36/36 agent calls (12 cases x 3 draws)
Evaluation	<code>inspect_ai</code> tasks in <code>evals/</code> , with a gate or target per task	93 tests, plus one eval task per claim above

Guard rails by design

- **Redaction before the model.** Client names and site addresses are replaced with placeholders before any text leaves the machine, then restored in the draft.
- **Read-only tools.** All four tools only read; none writes to a client system.
- **Data stays data.** Guidance passages reach the drafting prompt fenced off as quoted material, and tool descriptions state they are reference text, not instructions.
- **Output checked in code.** Every number in a draft must match the screening output, and every citation must be one of the passages supplied to the model. A sentence that does not match is dropped and listed in `warnings`.
- **Numbers come from code, not the model.** Screening is deterministic: the same input gives the same output.

Built for non-software staff

Setup is one config entry in Claude Desktop. Results come back as structured data plus a short text rendering, and where the host supports MCP Apps the exceedance table renders inline. A first session is three plain requests: *“Screen the results for site DEMO-03 against the residential criteria”*, *“What does the guidance say about the exceedances?”*, *“Draft the results section.”*

How it was built

I wrote the specification, the architecture and the decision log first, then built the server, the ingest pipeline, the tests and the evals with Claude Code. The repo holds all of it: **src/** (server, screening, retrieval, drafting, redaction), **ingest/**, **evals/**, **tests/** and **docs/**. Setup and eval commands are in the README.